TLP:CLEAR

OPEN-SOURCE INTELLIGENCE

TECHNICAL REPORT

TeamPCP Supply Chain Attack Campaign

Multi-Ecosystem Credential-Chaining Operation

CVE-2026-33634 | CanisterWorm | Trivy | Checkmarx | LiteLLM | Telnyx

Report Date:	28 March 2026
Version:	1.0 (Initial Public Release)
Classification:	TLP:CLEAR — Unlimited Distribution
Status:	ACTIVE — Campaign Ongoing
CVE Reference:	CVE-2026-33634 (CVSS 9.4)
CISA KEV:	Added 26 March 2026 — Due Date: 9 April 2026
Threat Actor:	TeamPCP (DeadCatx3 / PCPcat / PersyPCP / ShellForce)
Complied By:	Douglas Mun with AI assistance

Disclaimer

This report is classified TLP:CLEAR and is approved for unlimited distribution. It is based entirely on open-source intelligence (OSINT) gathered from publicly available vendor advisories, government advisories (CISA KEV), security research publications, and community reporting. No classified, proprietary, or non-public information has been used in its preparation.

The campaign is still evolving. Some technical details, victim counts, and threat actor relationships may change as new information emerges. Where primary sources do not confirm attribution or follow-on actor relationships, they are labelled as assessed or unverified rather than fact.

This document does not constitute legal advice, nor does it represent the official position of any government agency. Organisations should conduct their own risk assessments and engage appropriate legal and technical counsel before taking remedial action. References to specific vendors, products, or services are for informational purposes and do not imply endorsement or criticism.

1. Executive Summary

The March 2026 TeamPCP campaign is best understood as a credential-driven software supply chain compromise, rather than a conventional single-CVE exploit. The attacker first retained or regained privileged access to trusted release automation, then poisoned real software distribution channels: Trivy releases and GitHub Actions, Docker Hub images, Checkmarx GitHub Actions and OpenVSX plugins, npm packages carrying the CanisterWorm, PyPI packages including LiteLLM, and, later, Telnyx. In each phase, the attacker reused stolen secrets to pivot into the next ecosystem, turning trusted CI/CD and package-publishing paths into malware-delivery and credential-theft infrastructure.

The most important technical lesson is that this campaign attacked trust anchors: mutable GitHub Action tags, release bots, package publisher credentials, container registries, and developer install paths. The malware family harvested environment variables, runner memory, SSH keys, cloud credentials, Kubernetes tokens, Docker auth, Terraform state, and other sensitive artefacts; encrypted them with AES-256-CBC and RSA-4096; exfiltrated them to the attacker's infrastructure; and, in some variants, established persistence via systemd or attempted Kubernetes-wide propagation.

For organisations globally, the key risk is not just the direct use of a compromised package. It is possible that software vendors, system integrators, AI teams, managed service providers, or government contractors executed poisoned tooling in build environments and then propagated signed or otherwise trusted downstream artefacts into production. CISA added CVE-2026-33634 to the Known Exploited Vulnerabilities catalogue on 26 March 2026, with a federal remediation due date of 9 April 2026. Section 7 of this report provides a detailed impact-scoping comparison with SolarWinds and Log4j to accurately frame the severity and nature of this campaign.

CRITICAL ASSESSMENT

Campaign Status: ACTIVE and ONGOING as of 28 March 2026.

Estimated Impact: Over 1,000 SaaS environments confirmed affected (Mandiant CTO, 25 March). Projections of 5,000–10,000 from secondary reporting.

Collaboration: Secondary reporting speculates that TeamPCP is linked to LAPSUS$ and a ransomware group called Vect. No primary-source vendor or government confirmation of these partnerships has been identified. This should be treated as UNVERIFIED INTELLIGENCE, not a firm analytic conclusion.

Novel Techniques: First documented use of ICP blockchain canister as C2; first self-propagating npm worm in the wild; WAV steganography for payload delivery; AI-powered autonomous initial access via hackerbot-claw.

Formal Identifiers: CVE-2026-33634 (CVSS 9.4), CISA KEV, GHSA-69fq-xp46-6x23.

2. Background

2.1 The Open-Source Supply Chain as Attack Surface

Modern software development is fundamentally dependent on open-source software (OSS). Security scanning tools like Trivy, development libraries like LiteLLM, and communications SDKs like Telnyx are deeply embedded in enterprise CI/CD pipelines and production environments. These tools run with elevated privileges in build environments, have access to deployment credentials, and often execute automatically without human review. This trust relationship creates a high-value target for adversaries.

Docker's write-up on this incident is especially clear about a critical distinction: this was not fundamentally a dependency-graph problem but a credential-and-provenance problem, in which valid vendor credentials were used to push malicious content through legitimate channels. The usual defensive assumption that "official namespace equals safe" was destroyed because the attacker compromised real projects and real distribution channels, not typo-squatted packages.

2.2 Threat Actor: TeamPCP

TeamPCP (also tracked as DeadCatx3, PCPcat, PersyPCP, ShellForce, and CipherForce) is a cloud-native cybercriminal operation that has been active since at least December 2025. Multiple researchers associate it with consistent infrastructure across phases: the same RSA key pair, tpcp.tar.gz naming convention, and tpcp-docs-prefixed GitHub repositories used as dead-drop staging. The actor maintains Telegram channels (@Persy_PCP and @teampcp) and embeds the string "TeamPCP Cloud stealer" in payloads. Snyk, Wiz, Datadog, Endor Labs, and multiple other security research organisations have tracked the full campaign.

The motive profile is assessed as primarily credential theft, access expansion, and monetisable downstream compromise. The malware prioritises cloud credentials, CI/CD secrets, registry tokens, and Kubernetes access, which are valuable for follow-on intrusion, data theft, republishing, extortion, resale, and infrastructure abuse. Some phases also show performative or signalling behaviour: public taunting of victims, suppression of issues using botnet accounts, and visible campaign branding.

ANALYTIC NOTE: LAPSUS$ AND VECT COLLABORATION CLAIMS

Secondary reporting (SecurityWeek, The Hacker News, MrCloudBook) speculates on TeamPCP's collaboration with LAPSUS$ for extortion and an emerging ransomware group called Vect. However, no primary-source vendor advisory, government statement, or first-party security research firm has confirmed an operational partnership. These claims should be treated as UNVERIFIED INTELLIGENCE and monitored for corroboration, not cited as fact in operational assessments.

2.3 Precursor Events and Earlier CVE

The campaign follows an escalating pattern of CI/CD-focused supply chain attacks: SolarWinds (2020), Codecov (2021), ua-parser-js (2021), and tj-actions/changed-files (2025), which pioneered the tag-poisoning technique later used against Trivy.

Background context also includes CVE-2026-26189 (GHSA-9p44-j4g5-cfx5), a command-injection vulnerability in trivy-action discovered before the March campaign. While that is a separate issue, it illustrates that the Trivy GitHub Actions surface had prior known

weaknesses. The March 19 wave, however, was primarily a retained-credential and release-poisoning event, not exploitation of that specific CVE in isolation.

3. Comprehensive Timeline

The following timeline consolidates events from primary vendor advisories (Aqua Security, Checkmarx, LiteLLM, Telnyx, Docker, CISA) and validated security research (Wiz, Datadog, Sysdig, Snyk, Aikido, Microsoft, Endor Labs, JFrog, Socket, Palo Alto Networks, Arctic Wolf, SANS, Kaspersky, ARMO, StepSecurity, Semgrep, Trend Micro, SafeDep). Precise exposure windows are drawn from vendor-published data.

Date / Window	Event
Late 2025	TeamPCP begins operations, scanning for exposed Docker APIs, Kubernetes control planes, Ray dashboards, and Redis services across the UAE, Canada, South Korea, Serbia, the US, and Vietnam. Cyble profiles the group as a cloud-native exploitation platform.
20 Feb 2026	The hackerbot-claw GitHub account has been created. Its profile describes a vulnerability pattern index covering 9 classes and 47 sub-patterns. StepSecurity later documents this as an AI-powered autonomous attack tool built on OpenCLAW that systematically scanned repositories for exploitable GitHub Actions workflows.
27–28 Feb 2026	Phase 1 — Initial Access. hackerbot-claw exploits a misconfigured pull_request_target workflow (apidiff.yaml, present since October 2025 and flagged by Boost Security's poutine tool on 29 November 2025) in Aqua Security's Trivy repository. The bot exfiltrates a PAT (ORG_REPO_TOKEN) with write access to all 33+ repositories in the Aqua Security GitHub organisation. The attacker validates access by creating branches with emoji patterns, privatises the repo, deletes 178 releases, and publishes a malicious VS Code extension.
1 Mar 2026	Aqua Security discloses the first incident and begins credential rotation. Aqua later acknowledges the rotation was not atomic (not all credentials were revoked simultaneously), leaving a window in which refreshed credentials may also have been exposed. This is the root cause of the March 19 attack.
19 Mar 2026 ~17:43 UTC	Phase 2 — Supply Chain Injection. TeamPCP uses retained credentials to force-push 76 of 77 version tags in aquasecurity/trivy-action and all 7 tags in aquasecurity/setup-trivy to malicious commits. The compromised aqua-bot account triggers publication of malicious Trivy v0.69.4 to GitHub Releases, Docker Hub, GHCR, Amazon ECR, deb/rpm, and get.trivy.dev. The legitimate Trivy scan runs in parallel, masking the compromise. Aqua reports the core incident was identified and contained by ~20:38 UTC.
Trivy Exposure Windows	trivy binary v0.69.4: ~18:22 UTC to ~21:42 UTC on 19 March. trivy-action: ~17:43 UTC on 19 March to ~05:40 UTC on 20 March. setup-trivy: ~17:43 UTC to ~21:44 UTC on 19 March. (Per Aqua Security advisory.)
19–20 Mar 2026	44 Aqua Security GitHub repositories are renamed with the tpcp-docs- prefix and the description "TeamPCP Owns Aqua Security." Imposter commits spoof maintainer identities with forged author metadata.
20 Mar 2026 20:45 UTC	Phase 3 — CanisterWorm deployed on npm. Aikido Security detects a large-scale compromise: dozens of packages across multiple organisations receiving unauthorised patch updates with identical malicious code. The worm uses ICP blockchain canisters for C2. 28 packages are compromised within 60 seconds within the @EmilGroup scope. Affected npm namespaces include @EmilGroup, @opengov, @teale.io, @airtm, and @pypestream.
22 Mar 2026	Malicious Trivy Docker Hub images v0.69.5 and v0.69.6 were published without corresponding GitHub releases. Docker confirms that the affected pulls include v0.69.4, v0.69.5, v0.69.6, and the latest during the relevant window. Separately, researchers

Date / Window	Event
	observe TeamPCP deploying WAV steganography to deliver payloads in a Kubernetes wiper variant targeting Iranian systems.
23 Mar 2026	Phase 4 — Checkmarx compromised. Checkmarx discloses a compromise of ast-github-action and kics-github-action, as well as two malicious OpenVSX plugins (ast-results 2.53.0 and cx-dev-assist 1.7.0). Exposure windows: OpenVSX 02:53–15:41 UTC; KICS GitHub Action 12:58–16:50 UTC. Remediation was completed the same day. New C2 domain Checkmarx [.]zone activated. This proves the campaign has crossed from OSS-only projects into commercial AppSec tooling.
24 Mar 2026 10:39–10:52 UTC	Phase 5 — LiteLLM compromised on PyPI. Versions 1.82.7 and 1.82.8 were published using credentials stolen from LiteLLM's CI pipeline (which ran unpinned Trivy). v1.82.8 adds a .pth file that executes on any Python invocation, even without importing litellm. The official LiteLLM Proxy Docker image was NOT affected (it pinned dependencies). Attackers use 73 compromised developer accounts to post 88 bot comments in 102 seconds, suppressing community reporting. PyPI quarantines packages within approximately 3–5 hours.
25 Mar 2026	Mandiant CTO Charles Carmakal states that over 1,000 SaaS environments are actively dealing with this threat actor. CanisterWorm count expands to 135+ malicious package artefacts across 64+ unique npm packages.
26 Mar 2026	CISA adds CVE-2026-33634 to the Known Exploited Vulnerabilities (KEV) catalogue, with a federal remediation due date of 9 April 2026.
27 Mar 2026 03:51–04:07 UTC	Phase 6 — Telnyx compromised on PyPI. Versions 4.87.1 and 4.87.2 were published with malicious code hidden inside WAV audio files (steganography). C2 at 83[.]142[.]209[.]203:8080. Telnyx confirms the compromise was limited to the PyPI distribution channel; their production platform and APIs were not affected. PyPI quarantines the packages by 10:13 UTC.
28 Mar 2026	Campaign remains active and ongoing. Endor Labs and others assess additional targets expected to be available across PyPI, RubyGems, crates.io, and Maven Central. TeamPCP has standardised WAV steganography as a delivery technique.

4. Technical Details of the Campaign

This section provides a detailed technical analysis of each phase of the campaign, drawing primarily on research from Aikido Security, Wiz, Datadog Security Labs, Sysdig, Semgrep, Trend Micro, Microsoft, Snyk, Endor Labs, JFrog, Socket, and SafeDep. The level of detail is intended to support SOC hunting, incident response triage, and architectural review.

4.1 Campaign Mechanics: Credential Chaining

The core pattern was: compromise a trusted automation or publisher credential, push malicious artefacts through an official channel, steal more credentials from downstream users, then reuse those newly stolen secrets to compromise the next project. Each phase yielded credentials that unlocked the next target. This is not a single incident; it is a self-reinforcing credential-powered chain that crosses ecosystem boundaries. Datadog, Sysdig, Aqua, Docker, and LiteLLM all describe a pivot pattern built on stolen CI/CD pipelines and the publication of secrets, rather than a single isolated software defect.

4.2 Initial Access: hackerbot-claw AI Agent

The campaign's initial access vector was an AI-powered autonomous attack tool operating under the GitHub account hackerbot-claw (created 20 February 2026). Its GitHub profile described a vulnerability pattern index covering 9 classes and 47 sub-patterns, indicating a purpose-built automated scanner. StepSecurity documented hackerbot-claw as achieving code execution across multiple OSS repositories (including Microsoft, Datadog, and CNCF projects) and exfiltrating GITHUB_TOKEN values to hackmoltrepeat[.]com/moult and recv. hackmoltrepeat[.]com.

The bot specifically targeted pull_request_target workflow misconfigurations. When a GitHub Actions workflow uses pull_request_target, it runs with the base repository's secrets and permissions, but can be tricked into checking out and executing code from a fork's pull request. Trivy's repository had a vulnerable workflow (apidiff.yaml) since October 2025; Boost Security's poutine tool flagged it on 29 November 2025 — over three months before it was exploited. The bot opened PR #10254 with a legitimate-sounding branch name, modified the GitHub Actions setup, and extracted the ORG_REPO_TOKEN PAT, which had repo-scope (and likely admin-scope) access across all 33+ Aqua Security repositories. The token was used in at least 33 workflows. This was not a human at a keyboard; it was an automated system that could read YAML files, identify misconfigurations, craft convincing pull requests, and exfiltrate secrets autonomously.

4.3 Trivy / GitHub Actions Malware (Phase 2)

On 19 March 2026, beginning at approximately 17:43 UTC, TeamPCP used the retained credentials to execute a multi-pronged attack. The attacker made imposter commits spoofing maintainer DmitriyLewen (on aquasecurity/trivy) and Guillermo Rauch / rauchg (on actions/checkout), with forged author names, emails, timestamps, and commit messages. This is possible because Git's commit identity is self-declared — there is no cryptographic verification of author metadata by default.

4.3.1 GitHub Actions Tag Poisoning

The attacker force-pushed 76 of 77 version tags in aquasecurity/trivy-action (every tag from v0.0.1 through v0.34.2; only v0.35.0 was protected by GitHub's immutable releases) and all 7 tags in aquasecurity/setup-trivy to point to malicious commits. Any CI/CD pipeline that referenced these actions via a version tag automatically began running attacker-controlled code on its next execution, with no visible changes to release metadata or workflow files. Old tags without a "v" prefix (0.0.1–0.34.2) should be considered compromised; new v-prefixed tags (v0.0.1–v0.34.2) created post-remediation point to original legitimate commits.

4.3.2 The TeamPCP Cloud Stealer (GitHub Actions Payload)

Wiz's analysis describes the poisoned trivy-action and setup-trivy payload as a three-stage credential stealer that runs before the legitimate Trivy scan, so pipeline output looks completely normal:

- **Stage 1 — Runner Process Memory Scraping:** The payload identifies the GitHub Actions runner process by executing pgrep -f Runner.Worker and pgrep -f Runner.Listener. It then reads /proc/<pid>/mem to dump the Runner. Worker process memory, extracting secrets that match GitHub runner secret patterns. This technique captures secrets that may never appear in environment variables, filesystem paths, or configuration files — they exist only in memory during workflow execution. Microsoft's Defender hunt guidance provides specific CloudProcessEvents and DeviceProcessEvents queries targeting these process patterns.
- **Stage 2 — Filesystem Credential Sweep (50+ Paths):** The stealer searches a comprehensive list of credential locations across multiple cloud providers and development tools. AWS: ~/.aws/credentials, environment variables AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY, EC2 IMDS tokens via metadata service. GCP: service account JSON files, GOOGLE_APPLICATION_CREDENTIALS environment variable (the payload specifically runs cat $GOOGLE_APPLICATION_CREDENTIALS 2>/dev/null). Azure: service principal credentials. SSH: all private keys in ~/.ssh/. Git: credentials from ~/.gitconfig and credential helpers. GitHub: PATs from environment variables and CI/CD secrets. Kubernetes: kubeconfig files from ~/.kube/config, service account tokens, cluster API keys. Docker: ~/.docker/config.json (Docker explicitly warns this file is a high-value target). npm: ~/.npmrc and NPM_TOKEN environment variables. PyPI: ~/.pypirc and TWINE_PASSWORD. Additionally: Terraform state files, .env files, shell history, and cryptocurrency wallets.
- **Stage 3 — Encrypted Exfiltration with Fallback:** All harvested data is encrypted with AES-256-CBC using a random 32-byte session key. The session key is then encrypted with RSA-4096/OAEP using the attacker's embedded public key. Both are bundled into tpcp.tar.gz and exfiltrated via HTTP POST to scan. aquasecurity [.]org (a typosquatted domain using "securtiy" instead of "security") with the header X-Filename: tpcp.tar.gz. If the primary exfiltration fails, the malware executes a fallback mechanism: it uses the victim's GITHUB_TOKEN to create a public repository named tpcp-docs in the victim's GitHub organisation and uploads the encrypted stolen data there. The presence of a tpcp-docs repository in any organisation is therefore a confirmed indicator of successful credential exfiltration.

4.3.3 Malicious Trivy Binary (v0.69.4)

In parallel with the GitHub Actions tag poisoning, the attacker triggered Trivy's standard release machinery to publish a backdoored v0.69.4 binary to GitHub Releases, Docker Hub, GHCR, Amazon ECR, deb and rpm packages, and get.trivy.dev. The malicious binary launched legitimate Trivy scanning functionality alongside the malicious process, ensuring that developers and CI/CD pipelines saw normal scan results. At the same time, credentials were being exfiltrated in the background.

On non-GitHub-Actions environments (developer workstations, cluster-side scanners, container hosts), the binary attempted persistence by writing a Python dropper (sysmon.py) to ~/.config/sysmon/ and installing a systemd user service (sysmon.service) that polled the same ICP canister (tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io) used by CanisterWorm for follow-on payloads. This shared C2 infrastructure is a key attribution indicator linking the Trivy binary compromise to the subsequent npm and PyPI phases. Additional malicious Docker images, v0.69.5 and v0.69.6, were pushed to Docker Hub on 22 March without corresponding GitHub releases, exploiting the disconnect between source repositories and artefact registries.

DOCKER HUB: CACHE PERSISTENCE WARNING

Docker explicitly warns that even after malicious upstream content is removed, registry mirrors, artefact proxies (including Nexus and Artifactory), and local caches may still hold the compromised bytes. Docker advises searching local stores, registry mirrors, and artefact proxy caches for compromised Trivy digests. Docker also notes that mounting the Docker socket in a compromised Trivy container can lead to a root-equivalent compromise on the host.

4.4 CanisterWorm: Self-Propagating npm Worm (Phase 3)

CanisterWorm is the first publicly documented self-propagating worm in the npm ecosystem. Aikido Security researcher Charlie Eriksen first detected it on 20 March 2026 at 20:45 UTC, when dozens of packages across multiple organisations began receiving unauthorised patch updates containing identical malicious code. The worm uses a three-stage architecture: a Node.js postinstall loader, a persistent Python backdoor, and an ICP-hosted dead-drop C2 with remote payload rotation.

4.4.1 Stage 1: The Postinstall Loader (index.js)

The entry point is a malicious postinstall script in package.json that activates automatically during npm install. The entire execution is wrapped in a try/catch block, ensuring npm install succeeds normally on all platforms regardless of whether the malicious logic fires. The loader only activates on Linux systems with systemd. It performs four actions: (1) decodes an embedded base64-encoded Python backdoor; (2) writes it to ~/.local/share/pgmon/service.py; (3) creates a systemd user service unit file at ~/.config/systemd/user/pgmon.service with Restart=always, RestartSec=5, and After=default.target; and (4) runs systemctl --user daemon-reload, then enables and starts the service. No root privileges are required — the entire persistence chain operates within the user's systemd scope. All artefacts use PostgreSQL-themed naming (pgmon, pglog, .pg_state) to blend in on developer machines.

4.4.2 Stage 2: The Python Backdoor (service.py)

The persistent Python backdoor uses only standard library modules (urllib.request, os, subprocess, time), so there is nothing to install. Its behaviour is designed for stealth and sandbox evasion:

- **Initial sleep:** Waits 300 seconds (5 minutes) before any network activity, long enough to outlast most sandbox environments.
- **C2 polling:** Function g() contacts the ICP canister at tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io with a spoofed Mozilla/5.0 User-Agent. The canister does not serve malware directly; it returns a URL as plain text pointing to wherever the real binary is currently hosted. Polling occurs every ~50 minutes (3,000 seconds).
- **Payload delivery:** Function e() fetches the URL returned by the canister, saves the binary to /tmp/pglog, marks it executable (chmod 0o755), and launches it in a fully detached process (start_new_session=True). The URL is saved to /tmp/.pg_state to prevent re-downloading the same payload.
- **Kill switch:** If the URL contains youtube.com, the script skips execution. This is the canister's dormant state. The attacker arms the fleet by pointing the canister to a real binary, and disarms it by switching back to a YouTube link. At the time of discovery, the canister was returning a Rickroll YouTube link — the entire infected fleet was dormant but fully operational.
- **Payload rotation:** The ICP canister exposes three methods: get_latest_link (retrieves the current payload URL), http_request (serves the URL to the backdoor), and update_link (allows the attacker to rotate to a new payload). The attacker can therefore change what executes on all infected machines at any time, without republishing any npm package and without triggering any detection in the npm registry.

4.4.3 Stage 3: The Worm (deploy.js)

The deploy.js script transforms CanisterWorm from a backdoor into a self-propagating worm. It is a standalone tool that the attacker runs with stolen tokens to maximise blast radius. Aikido notes it appears to be entirely AI-generated ("vibecoded") with no obfuscation. The worm: (1) reads NPM_TOKENS (comma-separated) or NPM_TOKEN from environment variables, processing each independently; (2) calls the npm /-/whoami endpoint to resolve the associated username for each token; (3) uses the npm search API (/-/v1/search with maintainer:<username>, paginated in batches of 250) to enumerate every publishable package; (4) fetches the current latest version and increments the patch number; (5) fetches each target package's original README before publishing to preserve appearances; and (6) runs npm publish --access public --tag latest for each package with the malicious payload injected. In the @EmilGroup scope, 28 packages were compromised in under 60 seconds.

4.4.4 Worm Evolution: Self-Propagating Mutation

A subsequent mutation detected in @teale.io/eslint-config versions 1.8.11 and 1.8.12 incorporated the propagation functionality directly into the postinstall hook via a findNpmTokens() function. This eliminated the need for the attacker to run deploy.js with stolen tokens manually. Instead, the postinstall script itself installs the persistent backdoor. Immediately attempts to locate all npm tokens in the developer's environment and spawns the worm as a fully detached background process. This makes every compromised install an automatic and autonomous propagation vector. The detached: true and .unref() flags ensure the worm continues running even after npm install finishes.

4.5 Kubernetes Wiper — Geopolitically Targeted Destruction (Phase 3.5)

On 22 March 2026, Aikido Security discovered a new payload in the TeamPCP arsenal that uses the same ICP canister (tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0.io) documented in the CanisterWorm campaign. This variant adds geopolitically targeted destructive capability. The stager script downloads kubectl if it's not already available, then fetches kube.py from a Cloudflare tunnel domain (initially souls-entire-defined-routes.trycloudflare[.]com/kamikaze.sh, later split into separate stager and payload files).

4.5.1 Four-Path Decision Tree

The payload executes a simple but devastating decision tree based on two checks. First, it detects Kubernetes by checking for /var/run/secrets/kubernetes.io/serviceaccount or the KUBERNETES_SERVICE_HOST environment variable. Second, it identifies Iranian systems by reading /etc/timezone or querying timedatectl for Asia/Tehran or Iran timezone values, or checking the LANG environment variable for fa_IR locale:

Environment	Iran Detected	Action
Kubernetes	Yes	Deploy destructive DaemonSet host-provisioner-iran across all nodes, including the control plane. Container named kamikaze mounts host root filesystem at /mnt/host, deletes all top-level directories (find /mnt/host -maxdepth 1 -not -name mnt -exec rm -rf {} +), then force-reboots every node via chroot /mnt/host reboot -f.
Kubernetes	No	Deploy persistent DaemonSet host-provisioner-std across all nodes. Installs the CanisterWorm backdoor as a system-level service on every node via chroot into the host filesystem.
Non-Kubernetes	Yes	Execute rm -rf / --no-preserve-root (complete host destruction).
Non-Kubernetes	No	Exit silently. Nothing happens.

Both DaemonSets use a security context with hostNetwork: true, hostPID: true, privileged: true, and an operator: Exists toleration to ensure scheduling on every node, including tainted control-plane nodes.

4.5.2 Lateral Movement: SSH and Docker API Spread

A later variant adds network-based lateral movement. It parses SSH authentication logs (/var/log/auth.log and /var/log/secure) to extract successful login pairs (usernames and IPs), steals SSH private keys from ~/.ssh/, and uses them to spread to other machines on the local /24 subnet with StrictHostKeyChecking=no to suppress host key verification prompts. It also scans for exposed Docker APIs on port 2375 and creates privileged Alpine containers with the host root mounted at / to deliver the same payload. Both lateral movement methods carry the Iran timezone check — Iranian targets are wiped, while all others receive the persistent CanisterWorm backdoor. Cloudflare tunnel domains rotated across iterations, and the PostgreSQL-themed camouflage evolved: service names moved from pgmon to internal-monitor to pgmonitor, with install paths shifting accordingly.

4.6 LiteLLM PyPI Variant (Phase 5)

LiteLLM's official advisory confirms that versions 1.82.7 and 1.82.8 were compromised after credentials were stolen from the Trivy dependency used in LiteLLM's CI/CD scanning workflow, which pulled Trivy from apt without a pinned version. The compromised action exfiltrated the

PYPI_PUBLISH token from the GitHub Actions runner environment. Importantly, the official LiteLLM Proxy Docker image was NOT affected because it pinned dependencies.

4.6.1 Three-Stage Malware Design

Semgrep and Trend Micro describe the LiteLLM malware as more aggressive than the Trivy variant, with a three-stage design:

- **Orchestrator:** Injected into litellm/proxy/proxy_server.py (v1.82.7) and additionally as a .pth file litellm_init.pth (v1.82.8). The orchestrator decodes a base64-embedded payload and executes it. The .pth mechanism in v1.82.8 is particularly dangerous: Python's startup path automatically executes .pth files, meaning the malicious code runs on any Python invocation, even without importing litellm. Simply having Litellm installed triggers execution.
- **Harvester:** Collects credentials from the same comprehensive list as the Trivy stealer (AWS, GCP, Azure, Kubernetes, SSH, Docker, Terraform, .env, shell history), encrypts with AES-256-CBC + RSA-4096 OAEP, and POSTs the tpcp.tar.gz bundle to models.litellm[.]cloud.
- **Persistence and Kubernetes Lateral Movement:** If running in a Kubernetes environment with sufficient RBAC privileges, the malware creates privileged DaemonSets across all cluster nodes, mounts host filesystems via hostPath: /, and plants host-level backdoors. It also attempts to enumerate and dump all Kubernetes secrets cluster-wide (kubectl get secrets --all-namespaces -o json || true). This converts a single compromised pod into a full cluster compromise.

4.6.2 Bot-Driven Issue Suppression

When community members began reporting the compromise in GitHub issue #24512, the attackers deployed a botnet, posting 88 comments from 73 unique accounts within a 102-second window (12:44–12:46 UTC). Snyk analysis found 76% account overlap with the botnet used during the earlier Trivy disclosure. Using the compromised maintainer account (krrishdholakia), the attackers closed issue #24512 as "not planned" and made commits to unrelated repositories with the message "teampcp update." The community opened a parallel tracking issue (#24518) and continued discussion on Hacker News (324 points). The affected versions were on PyPI for approximately 3–5 hours before quarantine.

4.7 Checkmarx and OpenVSX Variant (Phase 4)

Checkmarx confirms the attacker force-pushed malicious tags to ast-github-action (before v2.3.33) and kics-github-action (before v2.1.20), and published two malicious OpenVSX plugins: ast-results-2.53.0.vsix and cx-dev-assist-1.7.0.vsix. The KICS payload used a setup.sh stealer tied to the C2 domain checkmarx[.]zone and fell back to creating a docs-tpcp repository with the victim's GITHUB_TOKEN if direct C2 failed — the same fallback pattern from the Trivy phase. Sysdig confirmed that the payload, encryption scheme, and tpcp.tar.gz naming convention were identical to those in the Trivy phase.

This phase is strategically critical because it demonstrates the campaign shifted from OSS-only volunteer projects to commercial AppSec tooling and IDE/plugin distribution. Enterprise developers consuming "official" Checkmarx extensions or CI actions were directly exposed. Trend Micro additionally identified a checkmarx-util-1.0.4 artefact delivered directly from checkmarx[.]zone/raw rather than the public npm registry, indicating the attacker was also

serving malicious packages from their own infrastructure alongside the hijacked official channels.

Datadog's timeline analysis places the Checkmarx compromise as the bridge to the LiteLLM incident: by 23 March, the attacker had demonstrated a recurring pattern targeting GitHub Actions, package registries, and developer tooling across multiple vendor environments. The 13-minute gap between LiteLLM v1.82.7 (10:39 UTC) and v1.82.8 (10:52 UTC) on 24 March indicates the attacker was actively iterating during the attack window, adding the more aggressive .pth injection vector in the second release.

4.8 Telnyx PyPI Variant (Phase 6)

Telnyx confirms compromised versions 4.87.1 (03:51 UTC on 27 March) and 4.87.2 (04:07 UTC) were limited to the PyPI distribution channel; their production platform and APIs were not compromised. This phase introduces a new delivery technique: live payload fetching through WAV audio file steganography.

4.8.1 WAV Steganography Delivery

The malicious code injected into telnyx/_client.py (74 lines added to the legitimate module) triggers at the module scope — it executes on import of telnyx, with no user action required. A helper function _d() decodes a 4,436-character base64-encoded payload variable _p. The execution forks a detached child process (creates a new session leader, breaking the process away from the parent so it persists after the importing application exits). The detached script downloads ringtone.wav from the C2 (http://83[.]142[.]209[.]203:8080/ringtone.wav), which is a valid WAV audio file with the malicious payload embedded in the audio frame data. The choice of a .wav file format, suited to the Telnyx SDK's telecommunications purpose, demonstrates the attacker's awareness of the target ecosystem. The payload is extracted from the WAV data and piped into a fresh Python interpreter via subprocess.run([sys.executable, "-"], input=payload).

4.8.2 Platform-Specific Behaviour

The Telnyx payload exhibits a strategic behavioural split by operating system:

- **Windows:** Downloads hangup.wav from the same C2, extracts a native binary, drops it as msbuild.exe in %APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\ (with an accompanying .lock file). This provides persistence across reboots via the Windows Startup folder, giving TeamPCP long-term repeatable access.
- **Linux/macOS:** Extracts and executes a credential harvester from ringtone.wav in-memory. The harvester collects credentials, encrypts them with the same AES-256-CBC + RSA-4096 OAEP scheme using the identical RSA public key from the LiteLLM compromise, and exfiltrates as tpcp.tar.gz via HTTP POST to 83[.]142[.]209[.]203:8080 with the header X-Filename: tpcp.tar.gz. Critically, there is no persistence mechanism on Linux/macOS: the temporary directory and all artefacts are recursively deleted after exfiltration. This is a deliberate stealth choice — extract credentials and leave no trace.

JFrog's analysis raises an additional concern: since the payload is fetched at runtime from a plain HTTP C2 (not HTTPS), any man-in-the-middle attacker could respond with their own ringtone.wav containing arbitrary payload. Unlike the XZ backdoor (which performed signature validation on downloaded payloads), the Telnyx variant performs no validation. This means the malicious versions remain dangerous even if the original C2 goes offline.

4.8.3 Attribution and Credential Chain

Endor Labs and Socket assess that the most likely credential theft vector was the LiteLLM compromise itself: TeamPCP's harvester swept environment variables, .env files, and shell histories from every system that imported litellm. If any developer or CI pipeline had both litellm installed and access to the telnyx PyPI token, that token was already in TeamPCP's hands. The three-day gap between LiteLLM (24 March) and Telnyx (27 March) aligns with the time required to sift through stolen credentials and select the next target. Attribution confidence is HIGH based on: identical RSA-4096 public key, identical AES-256-CBC + RSA OAEP encryption scheme, tpcp.tar.gz archive naming and X-Filename header, and consistent registrar (Spaceship, Inc.) and hosting provider (DEMENIN B.V.) across campaign domains.

4.9 Malware Evolution Summary

The campaign demonstrates clear iterative development across phases. Each wave introduced new capabilities while retaining the core credential-harvesting and encrypted exfiltration infrastructure:

Phase	New Capabilities Added	Persistence
Trivy (19 Mar)	Runner memory scraping; 50+ path sweep; encrypted exfil; GitHub repo fallback; legitimate scan runs in parallel	systemd user service (sysmon.py)
CanisterWorm (20 Mar)	First npm self-propagating worm; ICP blockchain C2; automated npm publish across scopes; README preservation	systemd user service (pgmon)
K8s Wiper (22 Mar)	Geopolitical targeting (Iran); DaemonSet cluster-wide deployment; SSH lateral movement; Docker API exploitation; WAV steganography (first use)	DaemonSets + systemd (internal-monitor)
Checkmarx (23 Mar)	Cross-vendor pivot; commercial AppSec tooling; OpenVSX IDE plugin poisoning; brand impersonation C2 domain	Same stealer + fallback pattern
LiteLLM (24 Mar)	.pth auto-execution on any Python invocation; K8s lateral movement via DaemonSets; bot-driven issue suppression (73 accounts, 88 comments, 102 seconds)	systemd + .pth file + K8s DaemonSets
Telnyx (27 Mar)	WAV steganography as a standard technique; live C2 payload fetching; no-persistence stealth on Linux; Windows Startup folder persistence; platform-specific split behaviour	Windows only (msbuild.exe); Linux ephemeral

4.10 CVEs, Advisories, and Formal Tracking

Identifier	Description	Status
CVE-2026-33634	Aquasecurity Trivy Embedded Malicious Code (CVSS 9.4, CWE-506)	CISA KEV — Due 9 Apr 2026
CVE-2026-26189 / GHSA-9p44-j4g5-cfx5	Trivy Action command injection (background context)	Prior issue
GHSA-69fq-xp46-6x23	Trivy ecosystem supply chain compromise	Critical
GHSA-cxm3-wv7p-598c	Trivy supply chain attack	Critical
PYSEC-2026-2	LiteLLM PyPI advisory	Critical

Identifier	Description	Status
SNYK-PYTHON-LITELLM-15762713	LiteLLM compromise (Snyk)	Critical
MSC-2026-3271	CanisterWorm (Mend.io tracking)	Critical
XRAY-957731	Telnyx PyPI compromise (JFrog)	Critical

5. Indicators of Compromise (IOCs)

This section consolidates IOCs from vendor advisories (Aqua, Checkmarx, LiteLLM, Telnyx, Docker), security research (Wiz, Datadog, Aikido, JFrog, StepSecurity, Microsoft, SafeDep, Semgrep, Trend Micro), and government advisories (CISA), where a source references additional non-public IOCs (e.g., Trend Micro's full 19-indicator table), which is noted.

5.1 Network, C2, and Exfiltration Infrastructure

Indicator	Type	Context / Phase
scan.aquasecurtiy[.]org	Domain (typosquat)	Trivy stealer primary exfiltration
45[.]148[.]10[.]212	IP Address	Trivy binary exfiltration
plug-tab-protective-relay.trycloudflare[.]com	Cloudflare Tunnel	Trivy rotating C2
tdtqy-oyaaa-aaaae-af2dq-cai.raw.icp0[.]io	ICP Canister	CanisterWorm/Trivy dead-drop C2 (blockchain)
checkmarx[.]zone	Domain (brand impersonation)	Checkmarx phase C2 and exfiltration
checkmarx[.]zone/raw	URL Path	Checkmarx raw payload delivery
checkmarx[.]zone/vsx	URL Path	OpenVSX stolen data exfiltration
models.litellm[.]cloud	Domain (brand impersonation)	LiteLLM phase C2
LiteLLM[.]cloud	Domain	LiteLLM infrastructure
83[.]142[.]209[.]203:8080	IP Address	Telnyx phase C2 / WAV delivery
83[.]142[.]209[.]11	IP Address	Cross-campaign infrastructure (LiteLLM reports)
46[.]151[.]182[.]203	IP Address	Cross-campaign infrastructure
souls-entire-defined-routes.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm stage payload
investigation-launches-hearings-copying.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm infrastructure
championships-peoples-point-cassette.trycloudflare[.]com	Cloudflare Tunnel	CanisterWorm infrastructure
hackmoltrepeat[.]com/molt	Domain	hackerbot-claw precursor — token exfiltration
recv.hackmoltrepeat[.]com	Domain	hackerbot-claw precursor — token exfiltration

5.2 File, Service, and Repository Artefacts

Artifact	Context
tpcp.tar.gz / X-Filename: tpcp.tar.gz	Encrypted exfiltration archive (campaign signature, all phases)
tpcp-docs / docs-tpcp / tpcp-docs-* repo pattern	Fallback exfiltration via GitHub repository creation
~/.config/sysmon/sysmon.py	Trivy binary Python dropper
~/.config/systemd/user/sysmon.service	Trivy binary systemd persistence
/tmp/runner_collected_*	Trivy GitHub Action temporary exfiltration staging
~/.local/share/pgmon/service.py	CanisterWorm Python backdoor
~/.config/systemd/user/pgmon.service	CanisterWorm persistence service
/tmp/pglog	CanisterWorm downloaded payload binary
/tmp/.pg_state	CanisterWorm payload URL state file
/var/lib/svc_internal/runner.py	Later-stage CanisterWorm/wiper variant
/etc/systemd/system/internal-monitor.service	Later-stage persistence (system-level)
/var/lib/pgmon/pgmon.py	Later-stage pgmonitor variant
/etc/systemd/system/pgmonitor.service	Later-stage persistence variant
litellm_init.pth / LiteLLM_init.pth	LiteLLM auto-execution on Python startup
litellm/proxy/proxy_server.py	LiteLLM backdoored module
ast-results-2.53.0.vsix	Malicious OpenVSX extension (Checkmarx)
cx-dev-assist-1.7.0.vsix	Malicious OpenVSX extension (Checkmarx)
checkmarx-util-1.0.4 / checkmarx-util-1.0.4.tgz	Artefact from checkmarx[.]zone (Trend Micro)
telnyx/_client.py	Telnyx backdoored module
%APPDATA%\...\Startup\msbuild.exe	Telnyx Windows persistent dropper
msbuild.exe.lock	Telnyx Windows lock file
~/.config/audiomon/audiomon.py	Telnyx Linux persistence variant
~/.config/systemd/user/audiomon.service	Telnyx Linux persistence variant
~/.docker/config.json	Target credential file (Docker explicit warning)

5.3 Affected Software Versions and Exposure Windows

Component	Compromised Versions	Safe Version	Exposure Window
aquasecurity/trivy (binary)	v0.69.4, v0.69.5, v0.69.6	v0.69.2, v0.69.3	v0.69.4: ~18:22–21:42 UTC 19 Mar
aquasecurity/trivy-action	76 of 77 tags (v0.0.1–v0.34.2)	@v0.35.0 (SHA: 57a97c7e)	~17:43 19 Mar – ~05:40 20 Mar
aquasecurity/setup-trivy	All 7 tags before the fix	v0.2.6 (rebuilt)	~17:43–21:44 UTC 19 Mar
Docker Hub trivy tags	0.69.4, 0.69.5, 0.69.6, latest	See digests below	19–22 Mar window
litellm (PyPI)	1.82.7, 1.82.8	1.82.6	~10:39 UTC 24 Mar, ~3–5 hrs
telnyx (PyPI)	4.87.1, 4.87.2	4.87.0	03:51–10:13 UTC 27 Mar
Checkmarx/ast-github-action	Before v2.3.33	v2.3.33+	23 Mar
Checkmarx/kics-github-action	Before v2.1.20	v2.1.20+	12:58–16:50 UTC 23 Mar
Checkmarx OpenVSX	ast-results 2.53.0, cx-dev-assist 1.7.0	Prior versions	02:53–15:41 UTC 23 Mar
66+ npm packages	CanisterWorm patch versions	Pre-compromise versions	20–21+ Mar

5.4 Hashes and Digests

5.4.1 Docker Image Digests for Compromised Trivy Tags (Docker official)

Tag	SHA-256 Digest
0.69.4	sha256:27f446230c60bbf0b70e008db798bd4f33b7826f9f76f756606f5417100beef3
0.69.5	sha256:5aaa1d7cfa9ca4649d6ffad165435c519dc836fa6e21b729a2174ad10b057d2b
0.69.6	sha256:425cd3e1a2846ac73944e891250377d2b03653e6f028833e30fc00c1abbc6d33

5.4.2 Malicious Trivy Binary Hashes (Wiz)

Platform	SHA-256
Linux-64bit	822dd269ec10459572dfaaefe163dae693c344249a0161953f0d5cdd110bd2a0
Linux-ARM64	e64e152afe2c722d750f10259626f357cdea40420c5eedae37969fbf13abbecf
macOS-ARM64	6328a34b26a63423b555a61f89a6a0525a534e9c88584c815d937910f1ddd538
macOS-64bit	e6310d8a003d7ac101a6b1cd39ff6c6a88ee454b767c1bdce143e04bc1113243
Windows-64bit	0880819ef821cff918960a39c1c1aada55a5593c61c608ea9215da858a86e349
Linux-32bit	f7084b0229dce605ccc5506b14acd4d954a496da4b6134a294844ca8d601970d

Platform	SHA-256
Linux-ARM	bef7e2c5a92c4fa4af17791efc1e46311c0f304796f1172fce192f5efc40f5d7
Linux-PPC64LE	ecce7ae5ffc9f57bb70efd3ea136a2923f701334a8cd47d4fbf01a97fd22859c
Linux-s390x	d5edd791021b966fb6af0ace09319ace7b97d6642363ef27b3d5056ca654a94c
FreeBSD-64bit	887e1f5b5b50162a60bd03b66269e0ae545d0aef0583c1c5b00972152ad7e073

5.4.3 Telnyx Package Hashes (SafeDep)

Version	SHA-256
telnyx==4.87.1	7321caa303fe96ded0492c747d2f353c4f7d17185656fe292ab0a59e2bd0b8d9
telnyx==4.87.2	cd08115806662469bbedec4b03f8427b97c8a4b3bc1442dc18b72b4e19395fe3

5.4.4 CanisterWorm Hashes by Wave (Aikido)

Component	Wave	SHA-256
index.js	Wave 1	e9b1e069efc778c1e77fb3f5fcc3bd3580bbc810604cbf4347897ddb4b8c163b
index.js	Wave 2	61ff00a81b19624adaad425b9129ba2f312f4ab76fb5ddc2c628a5037d31a4ba
index.js	Wave 3	0c0d206d5e68c0cf64d57ffa8bc5b1dad54f2dda52f24e96e02e237498cb9c3a
index.js	Wave 4	c37c0ae9641d2e5329fcdee847a756bf1140fdb7f0b7c78a40fdc39055e7d926
deploy.js	Wave 1	f398f06eefcd3558c38820a397e3193856e4e6e7c67f81ecc8e533275284b152
deploy.js	Wave 2	7df6cef7ab9aae2ea08f2f872f6456b5d51d896ddda907a238cd6668ccdc4bb7
deploy.js	Wave 3+	5e2ba7c4c53fa6e0cef58011acdd50682cf83fb7b989712d2fcf1b5173bad956

5.5 Kubernetes, Container, and Runtime Behavioural Indicators

Indicator	Type	Source
Pod naming pattern node-setup-*	K8s pod name	Datadog / Aikido
Container names: kamikaze, provisioner	K8s container	Datadog / Aikido
DaemonSet: host-provisioner-iran	K8s DaemonSet	Aikido (Iran wiper)
DaemonSet: host-provisioner-std	K8s DaemonSet	Aikido (non-Iran backdoor)
Outbound SSH with StrictHostKeyChecking=no	SSH lateral movement	Aikido
Outbound connections to port 2375 (Docker API)	Network scan	Aikido
Privileged Alpine containers with hostPath: /	K8s escape	Aikido
pgrep -f Runner.Listener	Process enumeration	Microsoft Defender
pgrep -f Runner.Worker	Process enumeration	Microsoft Defender

Indicator	Type	Source
kubectl get secrets --all-namespaces -o json || true	K8s secret theft	Microsoft Defender
Python-urllib* user agent	Network hunt lead	Microsoft Defender
Outbound HTTP POST with X-Filename: tpcp.tar.gz	Exfiltration header	All phases

6. Current and Potential Impact

6.1 Impact on the Open-Source Ecosystem

The campaign destroys the usual defensive assumption that “official namespace equals safe.” The attacker compromised real projects and real distribution channels. The malicious artefacts were not flagged by conventional vulnerability scanning because they were not vulnerable dependencies — they were legitimate release channels weaponised via stolen publisher credentials.

The mutable Git tag model was a core enabler. Over 10,000 GitHub workflow files reference trivy-action. Every one of them trusted that the tag they pinned still pointed to the code they reviewed. That trust was broken when TeamPCP rewrote 76 of 77 tags in under an hour. The .pth injection technique in LiteLLM v1.82.8 further expanded the attack surface: simply installing the package triggered execution on any Python invocation, even without explicit import.

6.2 Impact on Downstream Users and Commercial Software

Any runner, developer laptop, or build container that executed the poisoned artefacts may have exposed GitHub PATs, cloud service credentials, Kubernetes service account tokens, SSH keys, Docker registry auth, secrets managers, Terraform state, and environment variables. In containerised environments, Docker specifically warns that mounting the Docker socket into a compromised Trivy container can lead to a root-equivalent compromise of the host.

The campaign crossed from OSS into commercial AppSec tooling (Checkmarx), AI infrastructure (LiteLLM, which serves ~95 million monthly downloads and typically stores multiple LLM provider API keys), and telecommunications SDKs (Telnyx, with ~670,000 monthly downloads). This demonstrates a “downstream signing and trust amplification” problem: a compromised CI/CD runner can leak publisher keys that are then used to ship malicious updates from real vendor accounts into production channels.

6.3 Cache Persistence Risk

A secondary but operationally critical risk is cache persistence. Even after malicious upstream content is removed from registries, artefact proxy caches (Nexus, Artifactory), registry mirrors, Docker local stores, CI worker images, golden base images, and developer laptop caches may still hold the compromised bytes. Organisations must actively search these caches for the specific compromised digests and versions, rather than assuming that upstream remediation automatically propagates.

7. Impact Scoping: Comparison with SolarWinds and Log4j

This campaign should not be framed as “another SolarWinds” or “another Log4j.” It sits between them. It has a narrower software footprint than Log4j because it does not affect a near-universal library embedded across vast numbers of Java products. It is also less clearly strategic and less stealth-proven than SolarWinds, where trojanised Orion updates reached roughly 18,000 customers and a smaller, high-value subset was then selectively exploited for espionage. But it is still a serious event because it compromises trust anchors in modern software delivery: CI/CD actions, release automation, package publishing, and security tooling itself.

7.1 Compared with SolarWinds

The SolarWinds campaign compromised Orion, a network management platform deployed deep inside enterprise and government environments, where it naturally had privileged visibility into infrastructure. U.S. sources later assessed that while up to 18,000 customers received the tainted update, fewer than 100 were actually compromised through SUNBURST, underscoring that SolarWinds combined broad initial seeding with selective follow-on exploitation for espionage.

By contrast, the current TeamPCP wave is centred on developer ecosystems and build systems: malicious Trivy releases, poisoned GitHub Actions, compromised PyPI and npm packages, and downstream abuse of publishing credentials. That makes it less like a covert strategic implant in network management infrastructure, and more like a developer-trust compromise designed to steal secrets and pivot fast across the software supply chain. TeamPCP currently appears less optimised for long-dwell espionage inside national-level critical networks than SolarWinds was.

7.2 Compared with Log4j

NCSC described Log4Shell as affecting a widely used logging tool used by millions of computers worldwide, while CIS estimated the flaw was present in over 100 million instances globally. CISA and its partners described the exploitation of Log4j as active and widespread across consumer, enterprise, and even operational technology products. That is a very different exposure model from TeamPCP, where compromise depends on whether an organisation pulled a specific malicious version, referenced a poisoned mutable tag, or consumed a compromised package during a finite exposure window. In terms of sheer breadth, Log4j remains the larger class of events.

7.3 Why TeamPCP Is More Dangerous Than a Normal “Bad Package” Incident

TeamPCP is more dangerous than a normal supply chain incident in one specific way: it attacks the security and release machinery that other software depends on. Microsoft describes the Trivy phase as a CI/CD-focused supply chain attack that simultaneously compromised the scanner binary and associated GitHub Actions. At the same time, NVD notes that the actor used compromised credentials, force-pushed 76 of 77 trivy-action tags, and replaced all 7 setup-trivy tags with malicious commits. The campaign has since expanded to affect LiteLLM, Checkmarx, multiple npm namespaces, and Telnyx, demonstrating the ability to cascade across ecosystems. While it is not as universal as Log4j, it is more structurally dangerous than a single vulnerable library because it can turn trusted build paths into malware distribution paths.

7.4 Highest-Risk Victims

The highest-risk victims are not general end users. They are: upstream OSS maintainers and small maintainer teams; commercial software vendors whose developers or pipelines pulled affected artefacts; CI/CD administrators; developers with publish rights to PyPI, npm, Docker Hub, or OpenVSX; AI and platform teams using packages such as LiteLLM; and any organisation that builds, signs, scans, or redistributes software to others. In other words, the danger is concentrated on the people and systems that can amplify compromise downstream. That makes this campaign especially relevant to software vendors, MSSPs, system integrators, government contractors, and platform engineering teams, even when they are not themselves the original intended target.

7.5 Trivy Is Not Confined to Dev/UAT Environments

It would be too optimistic to characterise Trivy as a tool used only in development or UAT environments. Official Trivy documentation shows it can scan Kubernetes clusters directly. It can be installed as a Kubernetes Operator that continuously scans cluster resources (the Trivy Operator tutorial documents automatic scanning every six hours). Docker further warns that if a compromised Trivy image ran with the Docker socket mounted, the container effectively had root-level access to the host. The campaign is therefore not confined to developer laptops and test environments. It can also reach build runners, registry mirrors, artefact caches, cluster-side scanners, and container hosts, which are often much closer to production trust boundaries than a normal development workstation.

7.6 Direct vs Indirect Production Impact

For critical infrastructure and enterprise production environments, the most defensible judgment is that direct production infection is likely lower than with Log4j and probably lower than with SolarWinds, but indirect production impact is not low. The direct path is narrower because this campaign does not automatically expose every internet-facing Java service, and Trivy itself is not as deeply embedded in mainstream production application stacks as Log4j was. However, if a compromised build environment exposed cloud credentials, Kubernetes tokens, SSH keys, or registry credentials, the attacker may not need the package to exist in production at all. They can move from dev or CI into artefact repositories, cloud control planes, container registries, signing workflows, or deployment secrets.

CRITICAL SCOPING LANGUAGE

The right scoping language is NOT “production impact is limited.”

The stronger and more accurate wording is: “Initial compromise is concentrated in software development and build environments, but downstream production impact depends on the privilege and connectivity of those environments.”

Where CI/CD systems hold production cloud credentials, deployment tokens, registry push rights, or signing keys, the campaign can absolutely become a production incident even if the malicious scanner never runs on a production server.

Where build systems are isolated, use short-lived credentials, and are separated from production deployment authority, the blast radius is materially reduced.

NVD explicitly states that if compromised versions run, all secrets accessible to those pipelines should be treated as exposed and rotated immediately.

7.7 Bottom Line: Impact Scoping

SolarWinds was a stealthy, selective espionage operation seeded through a highly privileged monitoring platform. Log4j was a near-universal internet-scale software flaw that was hard to enumerate because it was buried deep in transitive dependencies. TeamPCP is neither of those exactly. It is a developer-ecosystem supply-chain compromise: narrower than Log4j in ubiquity, less strategically proven than SolarWinds in stealth and critical infrastructure penetration, but potentially devastating to organisations whose build and release systems hold powerful secrets or publish software to others.

FOR MOST ENTERPRISES

The primary risk is NOT mass production exploitation on day one.

It IS secret theft in CI/CD followed by downstream compromise of software, cloud, and deployment trust.

For critical infrastructure specifically, immediate direct exposure may be limited if affected artefacts were not used in production-facing paths. However, organisations with layered DevSecOps controls should not assume immunity — edge exposure through developer workstations, cached artefacts, contractor pipelines, or independently managed environments remains a residual risk that must be actively hunted for.

8. Threat Actor Profile and Assessed Motives

Attribute	Assessment
Designation	TeamPCP (aliases: DeadCatx3, PCPcat, PersyPCP, ShellForce, CipherForce)
Activity Period	December 2025 – Present (Active)
Classification	Cloud-native cybercriminal operation; hybrid model combining credential theft, access brokering, cryptomining, and destructive operations
Assessed Collaboration	UNVERIFIED: Secondary reporting speculates links to LAPSUS$ (extortion) and Vect (ransomware)—no primary-source confirmation identified as of 28 March 2026.
Communication	Telegram channels @Persy_PCP and @teampcp; public taunting of victims; campaign branding embedded in payloads
Attribution Confidence	HIGH (consistent RSA key pair, tpcp.tar.gz naming, tpcp-docs repo pattern, TeamPCP Cloud stealer string in payloads — per Snyk, Wiz, Datadog, Endor Labs)
Nationality	Unconfirmed; geographically diverse targeting suggests opportunistic exploitation rather than state-aligned operations
AI Capabilities	Uses hackerbot-claw, an AI agent built on openclaw, for automated vulnerability scanning and initial access across multiple OSS repositories
Novel Techniques	First ICP blockchain canister as C2; first self-propagating npm worm; WAV steganography; AI-powered autonomous initial access
Known Targeting (General)	UAE, Canada, South Korea, Serbia, US, Vietnam — opportunistic cloud infrastructure exploitation
Known Targeting (Destructive)	Iran — geopolitically targeted Kubernetes wiper (Asia/Tehran timezone, fa_IR locale)
Industry Focus	BFSI, Consumer Goods, Professional Services, DevOps/Security tooling vendors, AI/ML infrastructure

8.1 Assessed Motives

The motive profile is assessed as primarily credential theft, access expansion, and monetisable downstream compromise:

- **Credential Theft and Access Brokering:** The systematic harvesting of cloud credentials, API keys, publishing tokens, and Kubernetes access across multiple ecosystems suggests an access-brokering model where harvested credentials are sold, traded, or reused for follow-on intrusion.
- **Financial Gain:** Cryptomining (XMRig deployment on compromised hosts), and potential ransomware operations (if collaboration claims are confirmed).
- **Reputation and Signalling:** Public taunting, embedding campaign branding in payloads, defacement of compromised repositories, and suppression of community reporting all indicate a desire for recognition.
- **Geopolitical Targeting:** The Iran-targeted Kubernetes wiper introduces a geopolitical dimension. Whether this reflects state alignment, personal ideology, or client-directed operations remains under assessment.

9. Recommended Course of Actions

9.1 Immediate Actions (0–48 Hours)

- **Treat this as a credential exposure campaign first, package incident second.** Any environment that executed the named bad versions should have ALL reachable secrets rotated: GitHub PATs, cloud credentials (AWS, GCP, Azure), Kubernetes tokens, SSH keys, Docker registry auth, npm/PyPI publishing tokens, Terraform state, and environment variables. Rotation must be atomic.
- **Scope by exact artefacts and time windows.** Hunt for Trivy v0.69.4–0.69.6; trivy-action before v0.35.0; setup-trivy before v0.2.6; Docker tags 0.69.4/0.69.5/0.69.6/latest; LiteLLM 1.82.7/1.82.8; Checkmarx affected actions/plugins; Telnyx 4.87.1/4.87.2; and the named npm namespaces. Use the precise exposure windows in Section 5.3.
- **Deploy IOCs from Section 5 to SIEM and network monitoring.** Block all identified C2 domains, IP addresses, and ICP canister endpoints. Monitor for tpcp.tar.gz exfiltration headers, ICP beaconing to icp0.io, and outbound connections to the listed infrastructure.
- **Search caches and mirrors.** Check Nexus, Artifactory, registry mirrors, Docker caches, CI worker images, golden base images, and developer laptop caches for compromised digests (Section 5.4) or package versions. Upstream remediation does NOT automatically propagate to caches.
- **Search GitHub organisations for tpcp-docs / docs-tpcp repositories.** Their presence indicates successful credential exfiltration via the fallback mechanism.
- **On Linux systems, hunt for CanisterWorm persistence.** Check for pgmon. service, pgmonitor.service, internal-monitor.service, audiomon. service in the systemd user and system service directories. Check /tmp/pglog, /tmp/.pg_state, ~/.local/share/pgmon/.

9.2 Short-Term Actions (1–2 Weeks)

- **Pin all GitHub Actions to full, immutable commit SHA hashes.** Mutable version tags must not be used. This is the single most effective control against tag-poisoning attacks.
- **Pin container images to digests plus provenance verification.** Tag-based pulls are vulnerable to the same class of attack.
- **Pin Python/npm dependencies to vetted versions with hash verification.** Use pip --require-hashes, package-lock.json with integrity fields, and verify dependency integrity before installation.
- **Disable npm postinstall hooks by default.** Use npm config set ignore-scripts true globally. Selectively enable with @lavamoat/allow-scripts.
- **Conduct a full dependency audit** across all applications, including transitive dependencies. Pay particular attention to .pth files and postinstall hooks.
- **Restrict CI/CD runner network egress.** Implement allowlist-based outbound networking. Block IMDS or enforce IMDSv2 with hop limits.
- **Reduce secret blast radius in CI/CD.** Minimise default token scopes, use short-lived credentials, isolate release credentials from scan credentials, and make secret rotation atomic during incident response.
- **Enable runtime detection on CI runners.** Deploy Sysdig Secure, Falco, or equivalent to detect credential theft and data exfiltration from build environments.

9.3 Medium-Term Actions (1–3 Months)

- **Adopt SLSA Level 3+** across all critical CI/CD pipelines. Ensure provenance metadata is generated and verified for all artefacts.
- **Implement Trusted Publisher for PyPI** (OIDC-based authentication from CI/CD). Eliminate long-lived PyPI tokens.
- **Deploy ICP/blockchain C2 detection rules.** Monitor for DNS queries and connections to icp0.io domains. This technique is expected to be adopted by additional threat actors.
- **Instrument runtime and network detections.** Monitor for runner memory scraping (pgrep -f Runner.Worker), abnormal outbound POSTs from scanners, unexpected repo creation, unauthorised publishes, suspicious OpenVSX plugin installs, and privileged K8s DaemonSet creation.
- **Establish a dependency allowlisting framework—Centralise** vetting and approval of third-party OSS dependencies.
- **Conduct tabletop exercises** using the TeamPCP TTP playbook (credential chaining, tag poisoning, self-propagating worms, cache persistence) as the threat model.

10. Summary

This campaign is one of the clearest recent examples of how a compromise of trusted developer tooling can become a cross-ecosystem operation for credential theft and malware distribution. It started with Trivy, then spread into GitHub Actions, Docker Hub, npm, Checkmarx/OpenVSX, LiteLLM on PyPI, and Telnyx. The technical common thread is stolen trust: publisher credentials, bot accounts, mutable tags, CI secrets, and package-manager trust models.

For any organisation, the immediate priority is not only “do we use package X.” It is “did any trusted build, developer, or release environment execute one of these poisoned artefacts, and if so, what secrets, tokens, and downstream software releases were exposed afterwards?” That is the right lens for security teams, platform engineers, and executive leadership.

The fundamental lesson is that the open-source software supply chain is now a primary attack surface — not a peripheral risk. Organisations must treat dependency management with the same rigour applied to their own source code, pin dependencies to immutable references, actively search caches and mirrors for compromised artefacts, monitor for anomalous behaviour in build environments, and maintain rapid, atomic credential rotation capabilities.

CAMPAIGN STATUS AS OF 28 MARCH 2026

This campaign is ACTIVE and ONGOING. TeamPCP continues to expand to new targets and ecosystems. Additional compromises across PyPI, RubyGems, crates.io, and Maven Central are anticipated.

Cache persistence means that even “remediated” environments may still hold compromised artefacts. Active hunting is required.

All IOCs in this report should be treated as ACTIVE. Monitor cited sources for updates.

11. References

Sources are grouped by type. Primary vendor advisories are listed first, followed by government advisories, then security research. All URLs accessed on or before 28 March 2026.

#	Source	Title	URL
1	Aqua Security	Trivy Supply Chain Attack — What You Need to Know	https://www.aquasec.com/blog/trivy-supply-chain-attack-what-you-need-to-know/
2	Aqua Security / GitHub	Trivy Security Incident 2026-03-19 Discussion #10425	https://github.com/aquasecurity/trivy/discussions/10425
3	GitHub Advisory	GHSA-69fq-xp46-6x23 — Trivy Ecosystem Supply Chain Compromise	https://github.com/aquasecurity/trivy/security/advisories/GHSA-69fq-xp46-6x23
4	Docker	Trivy Supply Chain Compromise: What Docker Hub Users Should Know	https://www.docker.com/blog/trivy-supply-chain-compromise-what-docker-hub-users-should-know/
5	Checkmarx	Checkmarx Security Update	https://checkmarx.com/blog/checkmarx-security-update/
6	LiteLLM	Security Update March 2026	https://docs.litellm.ai/blog/security-update-march-2026
7	Telnyx	Python SDK Supply Chain Security Notice March 2026	https://telnyx.com/resources/telnyx-python-sdk-supply-chain-security-notice-march-2026
8	CISA	Adds CVE-2026-33634 to the Known Exploited Vulnerabilities Catalogue	https://www.cisa.gov/news-events/alerts/2026/03/26/cisa-adds-one-known-exploited-vulnerability-catalog
9	NVD	CVE-2026-33634 Detail	https://nvd.nist.gov/vuln/detail/CVE-2026-33634
10	Wiz	Trivy Compromised by TeamPCP — Supply Chain Attack	https://www.wiz.io/blog/trivy-compromised-teampcp-supply-chain-attack
11	Datadog Security Labs	LiteLLM and Telnyx Compromised on PyPI: Tracing the TeamPCP Campaign	https://securitylabs.datadoghq.com/articles/litellm-compromised-pypi-teampcp-supply-chain-campaign/
12	Sysdig TRT	TeamPCP Expands: Supply Chain Compromise Spreads from Trivy to Checkmarx	https://www.sysdig.com/blog/teampcp-expands-supply-chain-compromise-spreads-from-trivy-to-checkmarx-github-actions
13	Microsoft Security Blog	Detecting, Investigating, and Defending Against the Trivy Supply Chain Compromise	https://www.microsoft.com/en-us/security/blog/2026/03/24/detecting-investigating-defending-against-trivy-supply-chain-compromise/
14	Snyk	How a Poisoned Security Scanner Became the Key to Backdooring LiteLLM	https://snyk.io/articles/poisoned-security-scanner-backdooring-litellm/
15	Aikido Security	TeamPCP Deploys CanisterWorm on NPM Following Trivy Compromise	https://www.aikido.dev/blog/teampcp-deploys-worm-npm-trivy-compromise

#	Source	Title	URL
16	Aikido Security	TeamPCP Stage Payload CanisterWorm Iran	https://www.aikido.dev/blog/teampcp-stage-payload-canisterworm-iran
17	Aikido Security	Telnyx PyPI Compromised by TeamPCP CanisterWorm	https://www.aikido.dev/blog/telnyx-pypi-compromised-teampcp-canisterworm
18	JFrog Security Research	CanisterWorm — New Compromised Versions Detected	https://research.jfrog.com/post/canister-worm/
19	JFrog Security Research	TeamPCP Strikes Again — Telnyx Popular Library Hit	https://research.jfrog.com/post/team-pcp-strikes-again-telnyx-popular-library-hit/
20	Socket	CanisterWorm: npm Publisher Compromise	https://socket.dev/blog/canisterworm-npm-publisher-compromise-deploys-backdoor-across-29-packages
21	Mend.io	CanisterWorm: The Self-Spreading npm Attack	https://www.mend.io/blog/canisterworm-the-self-spreading-npm-attack-that-uses-a-decentralized-server-to-stay-alive/
22	Endor Labs	TeamPCP Isn't Done: LiteLLM's 95M Monthly Downloads	https://www.endorlabs.com/learn/teampcp-isnt-done
23	SafeDep	Malicious Telnyx PyPI Compromise	https://safedep.io/malicious-telnyx-pypi-compromise
24	Semgrep	TeamPCP Credential Infostealer Chain Attack Reaches LiteLLM	https://semgrep.dev/blog/2026/the-teampcp-credential-infostealer-chain-attack-reaches-pythons-litellm/
25	Trend Micro	Inside the LiteLLM Supply Chain Compromise	https://www.trendmicro.com/en/research/26/c/inside-litellm-supply-chain-compromise.html
26	StepSecurity	hackerbot-claw: AI-Powered Bot Exploiting GitHub Actions	https://www.stepsecurity.io/blog/hackerbot-claw-github-actions-exploitation
27	Palo Alto Networks	Breaking Down the Trivy Supply Chain Attack	https://www.paloaltonetworks.com/blog/cloud-security/trivy-supply-chain-attack/
28	Arctic Wolf	TeamPCP Supply Chain Attack Campaign	https://arcticwolf.com/resources/blog/teampcp-supply-chain-attack-campaign-targets-trivy-checkmarx-kics-and-litellm-potential-downstream-impact-to-additional-projects/
29	Kaspersky	Trojanization of Trivy, Checkmarx, and LiteLLM	https://www.kaspersky.com/blog/critical-supply-chain-attack-trivy-litellm-checkmarx-teampcp/55510/
30	SANS Institute	When the Security Scanner Became the Weapon	https://www.sans.org/webcasts/when-security-scanner-became-weapon
31	Cyble	TeamPCP Threat Actor Profile	https://cyble.com/threat-actor-profiles/teampcp/
32	ARMO	The Trivy Supply Chain Attacks Explained	https://www.armosec.io/blog/trivy-supply-chain-attack-ci-cd-security-lessons/

#	Source	Title	URL
33	Datadog IOCs (CSV)	TeamPCP Indicators of Compromise Feed	https://github.com/DataDog/indicators-of-compromise/raw/refs/heads/main/teampcp/iocs.csv
34	GitHub Advisory	GHSA-9p44-j4g5-cfx5 — Trivy Action Script Injection (CVE-2026-26189)	https://github.com/advisories/GHSA-9p44-j4g5-cfx5
34	U.S. GAO	SolarWinds Cyberattack Demands Significant Response (Infographic)	https://www.gao.gov/blog/solarwinds-cyberattack-demands-significant-federal-and-private-sector-response-infographic
35	SolarWinds	An Investigative Update of the Cyberattack	https://www.solarwinds.com/blog/an-investigative-update-of-the-cyberattack
36	NCSC (UK)	Log4j Vulnerability — What Everyone Needs to Know	https://www.ncsc.gov.uk/information/log4j-vulnerability-what-everyone-needs-to-know
37	Trivy Documentation	Kubernetes Scanning — Trivy Operator	https://trivy.dev/docs/latest/target/kubernetes/